

Material Didático — Dicionários e Arquivos em Python

Disciplina: Estrutura de Dados | **Nível:** Introdução à programação em Python

Material elaborado com linguagem simples, exemplos progressivos e questões práticas.

1. DICIONÁRIOS EM PYTHON

1.1 O que é um dicionário?

Um **dicionário (dict)** é uma estrutura de dados utilizada para armazenar informações no formato **chave** → **valor**. Podemos pensar em um dicionário como uma ficha de cadastro.

```
aluno = {  
    "nome": "Ana",  
    "idade": 45,  
    "cidade": "Campina Grande",  
    "curso": "Computação"  
}
```

Chave identifica a informação; **valor** é a informação armazenada.

1.2 Criando um dicionário

```
aluno = {  
    "nome": "Carlos",  
    "idade": 20,  
    "curso": "ADS"  
}  
  
print(aluno)
```

1.3 Acessando um valor

```
print(aluno["nome"])  
print(aluno["curso"])
```

Diferentemente de uma lista, o dicionário normalmente é acessado por meio de uma **chave**.

1.4 Adicionando informações

```
aluno["idade"] = 21  
aluno["cidade"] = "Campina Grande"  
print(aluno)
```

1.5 Alterando um valor

```
aluno = {  
    "nome": "Carlos",  
    "idade": 20  
}  
  
aluno["idade"] = 21  
print(aluno)
```

1.6 Removendo uma informação

```
aluno.pop("idade")
```

O método `pop()` remove uma entrada usando sua chave.

1.7 Verificando se uma chave existe

```
if "nome" in aluno:
    print("A chave nome existe!")
```

1.8 Obtendo todas as chaves

```
for chave in aluno.keys():
    print(chave)
```

1.9 Obtendo todos os valores

```
for valor in aluno.values():
    print(valor)
```

1.10 Obtendo chaves e valores

```
for chave, valor in aluno.items():
    print(chave, "→", valor)
```

1.11 Dicionário com notas

```
aluno = {
    "nome": "Maria",
    "nota1": 8.0,
    "nota2": 9.0
}

media = (aluno["nota1"] + aluno["nota2"]) / 2
print("Média:", media)

if media >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

1.12 Dicionário com vários alunos

```
alunos = {
    "Ana": {"idade": 20, "nota": 9},
    "Carlos": {"idade": 22, "nota": 7},
    "Maria": {"idade": 19, "nota": 8}
}

print(alunos["Ana"]["nota"])
```

Exercícios práticos — Dicionários

Questão 1 — Cadastro de aluno

Crie um dicionário chamado **aluno** contendo nome, idade, curso e cidade.

- Imprima o dicionário completo.
- Imprima apenas o nome.
- Imprima apenas o curso.

Questão 2 — Alteração

Utilize o dicionário da questão anterior.

- Altere a idade do aluno.
- Altere o curso.
- Imprima o dicionário novamente.

Questão 3 — Nova informação

Adicione ao dicionário as informações **email** e **telefone**. Depois imprima todas as informações.

Questão 4 — Remoção

Remova a informação **telefone** do dicionário e imprima o resultado.

Questão 5 — Verificação

Crie um programa que verifique se a chave **email** existe. Se existir, mostre “E-mail cadastrado.”; caso contrário, mostre “E-mail não cadastrado.”

Questão 6 — Percorrendo o dicionário

Crie o dicionário abaixo e utilize **for** e **items()** para apresentar cada chave e valor:

```
produto = {"nome": "Notebook", "preco": 2500, "marca": "Acer", "estoque": 10} + {"}
```

Questão 7 — Sistema de notas

Crie o dicionário abaixo, calcule a média e informe se o aluno foi aprovado (média ≥ 7) ou reprovado:

```
aluno = {"nome": "João", "nota1": 8, "nota2": 6} + {"}
```

Questão 8 — Cadastro de produtos

Crie um dicionário com 5 produtos e seus preços. Faça: a) mostrar todos os produtos; b) mostrar todos os preços; c) mostrar produto e preço; d) mostrar o produto mais caro.

2. ARQUIVOS EM PYTHON

2.1 Por que trabalhar com arquivos?

Durante a execução de um programa, dados armazenados apenas em variáveis são temporários. Arquivos permitem armazenar informações de maneira permanente, como alunos, notas, produtos e configurações.

2.2 Abrindo um arquivo

```
arquivo = open("dados.txt", "w")
```

O segundo parâmetro indica o modo de abertura.

Modo	Significado	Comportamento
r	Read	Leitura
w	Write	Escrita; substitui o conteúdo
a	Append	Adiciona ao final
r+	Read + Write	Leitura e escrita

2.3 Escrevendo em um arquivo

```
arquivo = open("dados.txt", "w")
```

```
arquivo.write("Ana\n")
arquivo.write("Carlos\n")
arquivo.write("Maria\n")
```

```
arquivo.close()
```

O `\n` representa uma quebra de linha.

2.4 Escrevendo números

```
arquivo = open("numeros.txt", "w")

for numero in range(1, 11):
    arquivo.write(str(numero) + "\n")

arquivo.close()
```

O método `write()` trabalha com texto. Por isso, números precisam ser convertidos com `str()`.

2.5 Lendo um arquivo inteiro

```
arquivo = open("dados.txt", "r")
conteudo = arquivo.read()
print(conteudo)
arquivo.close()
```

2.6 Lendo linha por linha

```
arquivo = open("dados.txt", "r")

for linha in arquivo:
    print(linha.strip())

arquivo.close()
```

O método `strip()` remove espaços e a quebra de linha no início ou no final do texto.

2.7 Adicionando conteúdo

```
arquivo = open("dados.txt", "a")
arquivo.write("Pedro\n")
arquivo.close()
```

O modo `a` mantém o conteúdo existente e acrescenta novos dados no final.

2.8 Diferença entre `w` e `a`

w: substitui o conteúdo anterior. **a:** adiciona ao final sem apagar o que já existe.

2.9 Utilizando `with`

```
with open("dados.txt", "r") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)
```

A estrutura `with` é recomendada porque o Python fecha o arquivo automaticamente ao final do bloco.

2.10 Exemplo completo — Cadastro de nomes

```
with open("nomes.txt", "w") as arquivo:
    arquivo.write("Ana\n")
    arquivo.write("Carlos\n")
    arquivo.write("Maria\n")

with open("nomes.txt", "r") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

2.11 Arquivo e lista

```
with open("nomes.txt", "r") as arquivo:
    nomes = arquivo.readlines()

nomes = [nome.strip() for nome in nomes]
print(nomes)
```

Assim, os dados do arquivo podem ser transformados em uma lista para processamento.

Exercícios práticos — Arquivos

Questão 1 — Primeiro arquivo

Crie um arquivo chamado **alunos.txt** e escreva nele os nomes de 5 alunos.

Questão 2 — Leitura

Leia o arquivo **alunos.txt** e imprima todos os nomes na tela.

Questão 3 — Linha por linha

Leia o arquivo **alunos.txt** e apresente os nomes um por vez, por exemplo: **Aluno: Ana**.

Questão 4 — Números

Crie **numeros.txt**, escreva os números de 1 a 20, um por linha, e depois leia e mostre os números.

Questão 5 — Append

Utilizando o arquivo da questão anterior: a) adicione os números de 21 a 30; b) leia o arquivo; c) mostre todos os números.

Questão 6 — Substituição

Crie **mensagem.txt**, escreva “Primeira mensagem” e depois, usando o modo **w**, substitua por “Nova mensagem”. Leia o arquivo e verifique o resultado.

Questão 7 — Cadastro de alunos

Crie um programa que permita ao usuário cadastrar nomes. Os nomes devem ser armazenados no arquivo **alunos.txt**.

Questão 8 — Lista de compras

Crie um programa que permita cadastrar 5 produtos em **compras.txt**. Depois leia o arquivo e apresente os produtos numerados.

Questão 9 — Notas dos alunos

Crie **notas.txt** e armazene dados no formato **Ana;8.5**. Depois leia o arquivo e mostre cada aluno e sua nota.

Questão 10 — Mini sistema

Crie um menu:

- 1 - Cadastrar aluno
- 2 - Listar alunos
- 3 - Sair

A opção 1 grava o nome no arquivo; a opção 2 lê e apresenta os alunos; a opção 3 encerra o programa.

3. DESAFIO FINAL — DICIONÁRIO + ARQUIVO

Agora vamos integrar os dois conteúdos: estruturas de dados e persistência de informações.

```
produto = {
    "nome": "Notebook",
    "preco": 2500,
    "quantidade": 5
}
```

Grave produtos em **produtos.txt** usando um formato simples, por exemplo:

```
Notebook;2500;5
Mouse;50;20
Teclado;100;10
```

Depois leia o arquivo e apresente cada produto com seu preço e quantidade.

4. ATIVIDADE DE FECHAMENTO — SISTEMA DE ALUNOS

```
=====
SISTEMA DE ALUNOS
=====
```

```
1 - Cadastrar aluno
2 - Listar alunos
3 - Pesquisar aluno
4 - Alterar aluno
5 - Remover aluno
6 - Sair
```

Cada aluno poderá ser representado por um dicionário:

```
aluno = {
    "nome": "Ana",
    "idade": 20,
    "curso": "ADS",
    "nota": 8.5
}
```

Os dados deverão ser armazenados em um arquivo.

Competências trabalhadas

- Criação e manipulação de dicionários.
- Chaves, valores, keys(), values() e items().
- Estruturas de repetição e decisão.
- Leitura e escrita de arquivos.
- read(), write(), readlines() e append.
- Uso de with open().
- Integração entre estruturas de dados e persistência.

Sequência didática sugerida

1. Trabalhar as questões de dicionários sem arquivos.
2. Trabalhar as questões de arquivos.
3. Finalizar com o desafio que integra dicionários e arquivos.
4. Estimular os alunos a transformar o desafio em um pequeno sistema de cadastro.

Observação: os exemplos foram mantidos intencionalmente simples para favorecer a compreensão dos conceitos fundamentais antes da introdução de estruturas mais avançadas.